

Project: Experimental Evaluation of Security Controls in Android-Backed APIs

Module: Advanced Computer Science Masters Project

Student Number: 24074199

Student Name: Kamal Prabhatkant Dubey

Supervisor Name: Anish Saini

Table of Content

Sr No	Title	Page No
1	Background Research and Literature Review	03
2	Summary of Progress to Date	07
3	Ethical, Legal, Professional and Social Considerations	09
4	Project Plan	13
5	Level of the Project	14
6	Referencing	15
7	Appendices	16

1. Background Research and Literature Review

1.1. Introduction

Application Programming Interfaces (APIs) server as the backbone of modern mobile applications, facilitating secure communication between client applications and backend services. In Android-based architecture, APIs handle authentication, authorization, data exchange, and business logic execution. While this architectural separation enhances scalability and modularity, it also introduces significant security risks, particularly when authentication and authorization mechanisms are improperly implemented.

Industry reports consistently identify Broken Object Level Authorization (BOLA), broken authentication, token management, and insufficient transport-layer protection as the most critical API security risks (OWASP, 2023). Although substantial research has examined API vulnerabilities, most prior work focuses on identifying or classifying vulnerabilities rather than quantifying the effectiveness of defensive security controls under controlled experimental conditions.

This project aims to address this gap by designing a controlled Android-based API testbed and experimentally evaluating how defensive mechanisms reduce exploit success rates. This section critically analyzes relevant literature to establish the theoretical and methodological foundation of the project.

1.2. Research Goals and Objectives

Research Goals:

To experimentally measure the effectiveness of authentication and authorization controls in preventing API exploitation in Android-based mobile architectures.

Research Question:

How effective are backend and Android-side security controls in mitigating API exploitation under controlled experimental conditions?

Objectives

- Design and implement a controlled Android + backend API testbed.
- Implement configurable JWT authentication and authorization controls.
- Execute structured attack scenarios under varying security configurations.
- Collect quantitative exploit success data.
- Analyse defensive effectiveness comparatively.

1.3. Background Research

Several key studies have informed this work:

Formal protocol analyses demonstrate that authentication systems are secure only under strict implementation assumptions. These works highlight that protocol design alone does not guarantee practical security: implementing correctness is critical.

Empirical API vulnerability research consistently identifies Broken Object Level Authorization (BOLA) as one of the most prevalent API weaknesses. This directly informs the inclusion of IDOR experiments within this project.

Android security research further shows that improper TLS validation and insecure token handling increase exploitation risk. This supports evaluation of client-side controls in addition to backend enforcement.

Unlike existing studies, which often classify systems as vulnerable or secure, this project introduces quantitative measurement of exploit success under controlled configuration changes.

1.4. Formal Authentication Protocol Analysis.

Authentication protocols such as OAuth 2.0 widely used in mobile-backend architecture. Fett, Kuster and Schmitz (2016) conducted a comprehensive formal security analysis of OAuth 2.0 using symbolic modelling techniques. The API flow under evaluation included authorization endpoints, token endpoints, and redirection mechanisms. The attacks analysed included token interception, session swapping, and cross-site request forgery.

Security was measured through formal proof of authentication and authorization properties within a formal model. The authors demonstrated that OAuth 2.0 is secure only when strict implementation assumption are upheld. Deviations in redirect URI validation or state parameter handling introduce exploitable vulnerabilities.

While methodologically rigorous, this approach evaluates theoretical correctness rather than empirical exploit likelihood. It does not quantify success reduction following defensive implementation changes.

Similarly, Somorovsky et al. (2011) investigated Single Sign-On systems using practical attack simulation. They demonstrated token misuse and signature verification bypass. However, systems were categorized as vulnerable or secure without probabilistic evaluation.

These studies establish the importance of correct implementation but lack quantitative comparative metrics. This limitation motivates the experimental design of the current project.

1.5. Empirical API vulnerability studies.

Large-scale empirical investigations have identified widespread API vulnerabilities. Georgiev et al. (2012) analysed TLS certificate validation routines in non-browser software. The APIs under examination were SSL/TLS validation libraries embedded within mobile and desktop clients. Using static code analysis, the authors discovered improper certificate verification logic enabling man-in-the-middle attacks. Measurement focused on the number of applications with flawed validation routines.

Haris et al. (2023) conducted dynamic testing of over 200 mobile backend APIs using fuzzing and request manipulation. APIs tested included authentication endpoints and resource-access endpoints. The primary attacks were Insecure Direct Object Reference (IDOR), privilege escalation, and broken authentication exploitation. Over 50% of tested APIs exhibited object-level authorization weaknesses.

Measurement in these studies involved vulnerabilities classification. APIs were labelled vulnerable if at least one exploit succeeded. However, repeated structured trails were not conducted to measure exploit probability.

Gupta et al. (2020) developed automated detection mechanisms for access control vulnerabilities in cloud APIs. Measurement focused on detection accuracy and vulnerability count.

Across these studies, exploit presence is identified, but exploit probability under varying defensive configurations is not measured.

1.6. Client-side and Android-Specific Security Research

Android client applications frequently contribute to API security weaknesses. Acar et al. (2017) demonstrated that developers often misuse certificate validation and security libraries due to incomplete understanding. Das et al. (2018) identified insecure token storage practices in Android applications.

These studies used static and dynamic code analysis to identify insecure implementations. However, none evaluated defensive improvement through structured comparative experimentation.

The present project extends this literature by integrating backend enforcement evaluation with Android client behaviour under controlled conditions.

1.7. Industry Frameworks and Risk Classification

OWASP (2023) ranks BOLA as the highest API risk. The OWASP Mobile Application Security Verification Standards (MASVS) provides guidance on secure mobile implementation. While these frameworks identify risks and recommend mitigations, they do not provide experimental measurement models.

This project operationalises these risk categories into experimentally measurable parameters.

1.8. Comparative Analysis of Measurement Approaches

Existing literature uses four primary measurement approaches:

1. Formal symbolic verification (Fett et al., 2016).
2. Vulnerability prevalence reporting (Georgiev et al., 2012)
3. Binary exploit demonstration (Gupta et al., 2020)
4. Dataset-wide vulnerability classification (Haris et al., 2023)

None of these methodologies incorporate repeated structured stack trails across controlled defensive configurations.

1.9. Application of Theory to Practice

Theoretical research has directly informed practical implementation decisions:

- Token expiration mechanisms are implemented to test replay resilience.
- Object-level authorization checks are designed based on access control theory.
- Layered security implementation reflects defense-in-depth principles.

For example, literature on Broken Object Level Authorization guided the implementation of user-bound resource validation within the backend. Rather than assuming correct authorization, the system allows toggling enforcement to measure the measurable difference in exploit feasibility.

This integration of theory and practical system design ensures the project contributes academically while solving real-world technical challenges.

1.10. Identified Research Gap

The literature demonstrates vulnerability existence but does not answer:

- How much does enabling a security control reduce exploit success?
- What is the probability of exploitation under specific configurations?
- How does layered defence influence exploit feasibility quantitatively?

Author	Year	API Type Tested	Methodology	Attacks Performed	Vulnerability found	Measurement Approach
Fett et al	2016	OAuth 2.0	Formal verification	Token interception	Session manipulation risk	Symbolic proof
Georgiev et al.	2012	TLS APIs	Static analysis	MITM	Certification validation bypass	Vulnerability count
Haris et al.	2023	Mobile REST API	Fuzzing	IDOR, privilege escalation	50% vulnerable	Binary classification
Gupta et al.	2020	REST APIs	Dynamic testing IDOR	IDOR	Missing auth checks	Exploit success
OWASP	2023	General APIs	Aggregated reports	BOLA, broken auth	Risk ranking	Industry ranking
Acar et al.	2017	Android apps	Empirical analysis	TLS misuse	Insecure cert validation	Vulnerability frequency
Chen et al.	2019	Web APIs	Dynamic analysis	Parameter tampering	Access control flaws	Exploit demonstration
Somorovsky	2012	OAuth implementation	Practical testing	Token misuse	Session swapping	Vulnerable
Calzavara	2020	OpenID APIs	Formal modeling	Code injection	Auth bypass	Model validation
Li et al.	2021	Mobile banking APIs	Dynamic testing	Replay attack	Weak expiry validation	Case study
Das et al.	2018	Android apps	Static analysis	Insecure storage	Token leakage	Code inspection
Zhou et al.	2020	Cloud APIs	Fuzzing	IDOR	Privilege escalation	Vulnerable classification
OWASP	2023	Mobile apps	Security guidelines	Auth bypass	Common weakness	Compliance checklist
Bansal et al	2022	REST APIs	Pentesting	Parameter tampering	Auth bypass	Exploit success
Rahman et al.	2021	Enterprise APIs	Risk assessment	Access misconfig	Broken ACL	Severity rating

2. Summary of Progress to Date

2.1. Literature Completion

A structured review of 20 academic and industry sources has been completed. Each paper was analysed based on:

- API type tested
- Attack vectors evaluated
- Measurement methodology
- Limitations

2.2. Experimental Architecture Design

The experimental system comprises:

- Android client layer
- Backend REST API layer
- JWT authentication mechanism
- Role-based access control
- Object-level authorization enforcement
- HTTPS communication layer
- Configurable security toggling

The system allows systematic toggling of authentication and authorization enforcement mechanisms.

Three experimental configurations have been defined:

1. Insecure Configuration
2. Partially Secure Configuration
3. Fully Secure Configuration

2.3. Defined Attack Parameters

The structured attack categories have been finalized:

- 2.3.1. Token Replay: Testing reuse of expired or captured authentication tokens (Li et al., 2021)
- 2.3.2. Insecure Direct Object Reference: Manipulating user identifiers withing endpoint requests to test object-level authorization (Haris et al., 2023)
- 2.3.3. Parameter Tampering: Altering request to attempt privilege escalation or unauthorized modification (Gupta et al., 2020).

2.4. Defined Measurement Metrics

The primary metric for evaluation is:

$$AttackSuccessRate = (SuccessfulExploit/TotalAttempts) \times 100$$

Each attack will be executed ten times per configuration, resulting in:

- 3 configurations x 10 attempts = 30 tests per attack
- 3 attack types = 90 total structured experiments

A secondary metric evaluates defensive improvement:

$$DefensiveEffectiveness = ((InsecureSuccess - SecureSuccess)/InsecureSuccess) \times 100$$

This allows quantitative comparison of defensive improvement.

2.5. Experimental Execution Plan:

For each attack scenario:

- 10 repeated attempts per configuration
- Across 3 configurations
- Total minimum test cases per attack – 30

Total planned experimental attempts:

- 3 attack types x 30 tests = 90 structured experiments

This ensures sufficient data for meaningful comparative analysis.

3. Ethical, Legal, Professional and Social Issues

3.1. Ethics Considerations

3.1.1. Requirement for Ethics Approval

This project does not require formal institutional ethics approval because:

- No human participants are involved.
- No surveys, interviews, or user studies are conducted.
- No personal or sensitive data is processed.
- All experimentation is performed within a self-developed and isolated environment.

However, the absence of formal approval requirements does not eliminate ethical responsibility. The project involves simulation of exploitation techniques, which demands strict adherence to responsible research practices.

3.1.2. Responsible Security Experimentation

The project involves implementing and testing offensive techniques such as:

- Token replay
- Insecure Direct Object Reference (IDOR)
- Parameter tampering

Although these techniques simulate malicious behaviour, experimentation is restricted to:

- A locally deployed backend API.
- Synthetic user accounts.
- Controlled attack simulations.
- No interaction with live or third-party systems.

No external applications, production systems, or publicly accessible APIs are tested.

This ensures compliance with responsible disclosure principles and avoids ethical misconduct.

3.1.3. Data handling and Privacy

The project uses only synthetic data generated within the experimental environment. No real user information is collected, stored, or processed.

If the project were extended to real-world systems or involved log analysis containing user-identifiable information, ethical approval and GDPR-complaint data governance would be mandatory.

3.1.4. Ethical Reflection on Offensive Research

Cybersecurity research often involves studying exploitation methods. Ethical risk arises when such research:

- Encourages misuse,
- Provided actionable attack blueprints,
- Or targets live systems without permission.

This project mitigates these risks by:

- Focusing on defensive evaluation rather than vulnerability exposure.
- Avoiding publication of exploit-ready code targeting real systems.
- Conducting controlled experimentation for academic purposes only.

Thus, the research promotes defensive strengthening rather than offensive capability proliferation.

3.2. Legal Considerations

3.2.1. Compliance with the Computer Misuse Act 1990

The UK Computer Misuse Act (1990) criminalises unauthorized access to computer systems. This project avoids legal violation because:

- No external systems are accessed.
- No live APIs are tested.
- All testing occurs with self-hosted environments.
- No bypass of authentication in real systems is attempted.

The research is confined to a legally owned and locally controlled environment.

3.2.2. Data Protection and GDPR Compliance

The General Data Protection Regulation (GDPR) applies to processing personal data. This project does not process:

- Real user credentials,
- personally identifiable information,
- or external datasets.

Therefore, GDPR compliance obligations do not apply at present.

However, if the project were scaled to include:

- Real API traffic,
- Log monitoring of production systems,
- Or vulnerability scanning of third-party services,

Then strict GDPR-compliant data handling procedures would be required.

3.2.3. Intellectual Property and Licensing

The project uses open-source technologies such as:

- FastAPI
- Python libraries
- Android SDK components

All tools are used in compliance with their respective open-source licenses.

No proprietary source code is analysed without permission.

3.2.4. Legal risk if Expanded

If this research were extended to test publicly available APIs without authorization, it could violate:

- Computer Misuse Act
- Service provider terms of use

- International cybercrime regulations

The project explicitly avoids such activity.

3.3. Professional Considerations

3.3.1. Adherence to Industry Standards

The project aligns with professional cybersecurity standards including:

- OWASP API Security Top 10
- OWASP Mobile Application Security Verification Standard (MASVS)
- Defence-in-depth Principles
- Responsible vulnerability research practices

The methodology emphasises:

- Controlled experimentation
- Reproducibility
- Transparent documentation

3.3.2. Research Integrity

To maintain academic integrity, the project:

- Properly references all research sources.
- Avoids plagiarism.
- Documents methodology transparently.
- Reports limitations honestly.

All experimental results will be reproducible and not selectively reported.

3.3.3. Quality Assurance

Professional standards require systematic testing and validation. The project incorporates:

- Repeated attack trials.
- Structured configuration comparison.
- Quantitative metric calculation.
- Controlled variable isolation.

This ensures scientific reliability rather than anecdotal findings.

3.4. Social Considerations

3.4.1. Impact on Privacy

API vulnerabilities often result in:

- Data breaches
- Exposure of Personal information
- Financial fraud

By quantitatively measuring defensive effectiveness, this research contributes to improving API resilience and protecting user privacy.

3.4.2. Security vs Accessibility Trade-offs

Security controls such as strict token validation and role enforcement may introduce usability overhead. Excessively restrictive mechanisms may impact performance or developer flexibility.

This project indirectly evaluates the cost-benefit balance of defensive enforcement by measuring effectiveness rather than assuming universal benefit.

3.4.3. Broader Societal Impact

Mobile APIs underpin:

Banking systems

Healthcare platforms

Government services

E-commerce platforms

Improved defensive implementation can reduce systemic risk in digital infrastructure.

By shifting the focus from vulnerability discovery to defensive effectiveness measurement, this project contributes to strengthening real-world API ecosystems.

4. Project Plan

4.1. Remaining tasks

Task	Description	Target Completion
Complete Android Client integration	Finalise communication layer	Week 01
Implement full security toggling	Final configuration states	Week 02
Conduct 90 structured experiments	Execute attack scenarios	Week 03
Data analysis and comparison	Compute metrics and graphs	Week 04
Final dissertation writing	Complete chapters 5-7	Ongoing

4.2. Project Management Approach

A hybrid structured-iterative approach is applied:

- Clear milestone planning
- Weekly progress review
- Supervisor feedback integration
- Risk monitoring (scope creep, timeline compression)

Time allocation includes dedicated analysis and writing phases to maintain academic rigor.

5. Level of the Project

This project qualifies as MSc-level due to:

- Integration of theory and controlled experimentation.
- Design of configurable security architecture.
- Quantitative evaluation methodology.
- Comparative defensive analysis.
- Structured validation and repeatability.

The artefact (Android-backed API testbed) is not the end goal but serves as an experimental instrument enabling measurable security evaluation.

The project demonstrates:

- Analytical depth
- Technical implementation capability.
- Experimental design rigor.
- Critical evaluation of results.

This exceeds simple application development and aligns with advanced cybersecurity research standards.

6. Referencing

Acar, Y. et al. (2017) 'The Impact of Information Sources on Code Security', IEEE Symposium on Security and Privacy.

Das, T. et al. (2018) 'A Large-Scale Study of Android Application Security', ACM CCS.

Fett, D., Küsters, R. and Schmitz, G. (2016) 'A Comprehensive Formal Security Analysis of OAuth 2.0', ACM CCS.

Georgiev, M. et al. (2012) 'The Most Dangerous Code in the World: Validating SSL Certificates in Non-Browser Software', ACM CCS.

Gupta, A. et al. (2020) 'Authorization Weaknesses in REST APIs', Journal of Web Engineering.

Haris, M. et al. (2023) 'Large-Scale Measurement of Mobile API Vulnerabilities', IEEE TDSC.

Li, X. et al. (2021) 'Replay Attacks in Mobile Banking APIs', Computers & Security.

OWASP (2023) API Security Top 10. Available at: <https://owasp.org>.

OWASP (2023) Mobile Application Security Verification Standard (MASVS). Available at: <https://owasp.org>.

Somorovsky, J. et al. (2011) 'On Breaking SAML', USENIX Security Symposium.

Zhou, Y. et al. (2020) 'Automated Detection of Access Control Vulnerabilities in Cloud APIs', IEEE Security & Privacy Workshops.

7. Appendices

Appendix 01 – Research Gap Alignment Matrix

Literature Insight	Identified Gap	Project Response
OAuth secure in theory	Lacks empirical validation	Controlled testbed
API BOLA common	No quantitative control testing	IDOR success rate measurement
TLS misuse frequent	Limited effectiveness measurement	HTTPS enforcement comparison
Android misconfiguration	Client + backend rarely integrated	Combined evaluation

Appendix 02 – Defined Attack Parameter Table

Attack Type	Parameter Under Test	Configuration Variation	Measurement Metric
Token Replay	Token expiry	Enabled vs Disabled	Success %
IDOR	Object-level auth	Enabled vs Disabled	Unauthorized access rate
Parameter Tampering	Role validation	Enabled vs Disabled	Escalation success
HTTPS Interception	TLS enforcement	HTTP vs HTTPS	Request acceptance

Appendix 03 – Experimental Configuration States

Configuration	Token Expiry	Object Auth	Role Check	HTTPS
Insecure	Off	Off	Off	Off
Partial	On	Off	On	On
Secure	On	On	On	On